Project Documentation

1. Introduction

Project Title: ShopSmart Your Digital Grocery Store Experience
Team Leader: Panneni Jyothsna Priya
Team Member : Paula Gavini

2. Project Overview

Purpose: Designed as a guided project (SmartInterns) demonstration to teach developers how to build an intuitive, feature-rich online grocery shopping experience—from frontend navigation to backend controls.

- User scan browse grocery items through a clean, interactive UI.
- Admin panel enables adding, editing, or removing products.
- Includes search and filter options for better product discovery.

3. Architecture

Frontend :

Framework: Built with React js, along with standard JavaScript HTML, and CSS
Structure: Organized under /src/ into folders like components/, admin_components/, and context/. The UI includes pages for Browse products, a responsive layout, search/filter features, and an admin panel to add/edit/remove items.
Role: Handles the user interface for customers and admins, displays products, and manages user interactions (cart, search, admin tasks).

Backend:

Runtime & Framework: Powered by Node.js with Express js.
Architecture: The central entrypoint is index.js. Database logic resides in db/ (connection, models, schemas in connect.js, products.js, schemas.js). It provides a RESTful API with endpoints to fetch products, search, filter, and perform admin CRUD (Create, Read, Update, Delete) operations.
Role: Hosts API endpoints, communicates with MongoDB, enforces business logic, and handles admin/product operations.

Database:

Uses MongoDB for data storage.
Likely employs Mongoose (or similar) schemas to define collections for products and possibly users/orders.
Role: Stores core data (product details, categories, possibly user/cart info) in a flexible NoSQL format.

4. Setup Instructions

Prerequisites:

- Install Node.js (v14+, includes npm).
- Have a MongoDB instance (local or MongoDB Atlas).

Installation :

Clone the repository:

```
git clone https://github.com/GaviniPaula/ShopSmart-Your-Digital-Grocery-Store-Experience.git
cd ShopSmart-Your-Digital-Grocery-Store-Experience
```

Install dependencies:

```
cd client
npm install
cd ../server
npm install
```

Configure environment variables:

```
In the frontend folder (create .env):
VITE_API_URL=http://localhost:5000
In the backend folder (create .env):
PORT=5000
MONGO_URI=your_mongodb_connection_string
JWT_SECRET=your_jwt_secret
PAYPAL_CLIENT_ID=your_paypal_client_id
CLOUDINARY_CLOUD_NAME=your_cloudinary_name
CLOUDINARY_API_KEY=your_cloudinary_api_key
CLOUDINARY_API_SECRET=your_cloudinary_secret
```

(Optional) Seed the database:

```
cd server
npm run seeder
```

Access locally:

- Frontend: http://localhost:3000
- Backend API: http://localhost:5000/api

5. Folder Structure

Client: (React Frontend)

- client/ or frontend/
- public/
- index.html
- src/
- components/: Reusable UI components
- pages/: Page-level views
- contexts/: React Contexts for global state management
- data/: Optional: mock or seed data
- App.js or App.tsx: Main app wrapper defining routes/layout
- index.js or main.tsx: Entry point rendering into DOM

Server: (Node.js + Express Backend)

- server/ or backend/
- models/: Mongoose schema definitions
- routes/: Express routes
- controllers/: Handlers/service logic for each route
- scripts/ or data/: (Optional) Database seeding files
- db/: DB connection and setup
- server.js or index.js: Entry point, sets up Express app
- package.json

6. Running the Application

Start backend server:

```
cd server
npm start
```

Start frontend server:

```
cd ../client
npm start
```

7. API Documentation

Product Endpoints

GET /api/products
Description: Get all products
Method: GET
Response:

```
[
  {
    "_id": "123",
    "name": "Fresh Apples",
    "price": 120,
    "image": "/images/apple.jpg"
  }
]
```

GET /api/products/:id
Description: Get product by ID
Method: GET
Params: id - Product ID
Response:

```
{
  "_id": "123",
  "name": "Fresh Apples",
  "price": 120,
  "description": "Fresh and crispy apples"
}
```

POST /api/products
Description: Add a new product (Admin only)
Method: POST
Body:

```
{
  "name": "Banana",
  "price": 80,
  "description": "Organic bananas",
  "image": "/images/banana.jpg"
}
```

Response: Created product object

PUT /api/products/:id
Description: Update product by ID (Admin only)
Method: PUT
Body: Same as POST
Response: Updated product object

DELETE /api/products/:id
Description: Delete product by ID (Admin only)
Method: DELETE
Response:

```
{ "message": "Product removed" }
```

8. Authentication

Authentication and authorization in this project are handled using JSON Web Tokens (JWT).

How it works:

- When a user logs in or registers, the server generates a JWT that includes the user's ID and role (if applicable).
- The token is signed with a secret key defined in the .env file (JWT_SECRET).
- This token is sent to the client and stored (typically in localStorage or cookies).

Request Authorization :

For protected routes (e.g., adding or deleting products), the client must include the token in the Authorization header:

```
Authorization: Bearer <your_token_here>
```

The backend middleware (authMiddleware.js or similar) verifies the token and grants access based on user role.

Role-based Access

- Basic users can view and purchase products.
- Admin users can create, update, and delete products.

9. User Interface

This section provides a description of the application's user interface and examples of key features.

- Homepage with product listings: The main page displaying various grocery items available for purchase.
- Login/Register screens: Modals or pages for user authentication, including fields for email, password, and registration details.
- Adding items to cart: The interactive process of selecting products and adjusting quantities, with the cart icon dynamically updating to reflect the number of items.
- Cart summary: A sidebar view of the cart, displaying the list of selected items, quantities, and the subtotal.
- Checkout process: The workflow for finalizing an order, including entering delivery details and confirming the purchase.
- Order confirmation: A pop-up confirming the successful placement of an order, including the total order value and expected delivery time.

10. Testing

Testing Strategy:

This project uses manual and automated testing methods to ensure reliability and performance.

Tools Used:

- Jest: For running JavaScript unit tests (commonly used with React).
- React Testing Library: For testing React components.
- Supertest: For testing Express js API endpoints.
- Postman: For manual API testing.

Frontend Testing:

Components are tested using React Testing Library and Jest
Common tests include rendering UI elements, form validation, and route handling.

Backend Testing:

API endpoints are tested using Supertest with Jest
Includes tests for product creation, retrieval, updating, and deletion.

Running Tests

Run the following command in respective folders:

Frontend :

```
npm test
```

Backend:

```
npm test
```

11 Screenshots or Demo

A comprehensive video demo showcasing the application's core functionalities is available.
VIDEO DEMO LINK: View Demo

12. Known Issues

- Cart persistence: Items in the cart are lost upon page refresh unless session storage or a database is used.
- Mobile responsiveness: Some pages may not be fully optimized for mobile screens
- Authentication edge cases: No password reset or token expiration handling yet.
- Admin validation: Admin-only routes can be accessed via direct API calls if the token is leaked.
- Error handling: API error responses are not fully standardized; improvements needed in backend error messages
- Limited test coverage: Not all routes/components are covered by automated tests.

13. Future Enhancements

- Persistent Cart: Store cart items in local storage or sync with database for session retention.
- Responsive Design: Fully optimize all views for mobile and tablet screen sizes
- Order History: Allow users to view their past orders and invoices.
- Password Reset & Token Expiry: Implement email-based password recovery and token expiration logic.
- Admin Features: Add inventory tracking, sales analytics, and customer management tools.
- Delivery Tracking: Integrate with shipping APIs for real-time delivery updates.
- Improved Testing: Achieve higher code coverage with unit, integration, and E2E tests.
- Multi-language Support: Add internationalization (i18n) for wider audience reach.
- Wishlist & Save for Later: Let users bookmark products they intend to buy later.
- Analytics Dashboard: Add graphical insights for admins (sales, active users, etc.).